Contours & Shape Analysis

Week 9 — Session 1: Finding contours & basic measurements

Practical Image Processing

Lecture 9.1 ▪ 60–70 min

Recap & today

- Wk8: segmentation gave us separated regions/blobs — a clean binary mask.
- Today (Session 1): turn that mask into a list of **objects**. We trace **contours** and take the first, most useful **measurements**: area, perimeter, bounding box, centroid.
- Session 2 goes on to shape *descriptors* and invariant matching.

DocScan milestone

Find each digit's contour and crop it by its bounding box — the step that hands clean, ordered digits to the reader (Week 13).

Roadmap

From regions to objects

Basic measurements

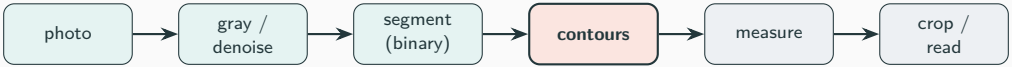
Boxes $\rightarrow$ crop, order, and filter

In-class exercise

Wrap-up

From regions to objects

Where we are in the pipeline



- A binary mask is still just *pixels*. We want a list of **things**: "9 digits, here are their positions and sizes."
- Contours are the bridge from a pixel grid to countable, measurable objects.

A contour = the outline of a region



binary digits



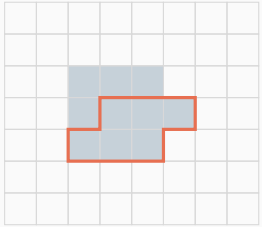
contours (red)

- A **contour** is an ordered list of the boundary points of one connected white region.
- From a clean binary image, `findContours` walks around each object and records its outline.
- Each contour is one object you can measure, filter, and crop.

Key idea

One binary mask $\rightarrow$ a Python list of contours, one entry per object.

How border-following works (intuition)



1. Scan the mask until you hit the first white pixel with a black neighbour — a boundary pixel.
2. "Walk" around the region, always keeping the object on one side (Moore / Suzuki tracing).
3. Record each boundary pixel in order until you return to the start.

Ordered, not a set

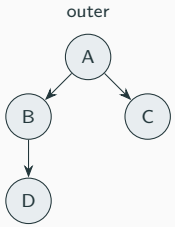
The output is a *sequence* of points, so "next point along the boundary" is meaningful.

findContours: modes & approximation

```python
contours, hierarchy = cv2.findContours(binary,
    cv2.RETR_EXTERNAL,        # only outer contours (ignore holes)
    cv2.CHAIN_APPROX_SIMPLE)  # compress straight runs to their endpoints
```

- **RETR_EXTERNAL**: outer boundaries only. **RETR_TREE**: full nesting hierarchy (Session 2).
- **CHAIN_APPROX_NONE**: every boundary pixel. **CHAIN_APPROX_SIMPLE**: only corner points — a rectangle becomes 4 points, not hundreds.
- Input must be a *binary* image; `contours` is a list of $N \times 1 \times 2$ point arrays.

Retrieval modes at a glance



RETR_TREE: full parent/child tree

- **RETR_EXTERNAL**: keep A only (drop nested B, C, D).
- **RETR_LIST**: all contours, *no* hierarchy.
- **RETR_CCOMP**: two levels (outer + holes).
- **RETR_TREE**: the complete nesting relationship.

✓ Good practice

For cropping isolated digits, **RETR_EXTERNAL** is the simplest correct choice.

Contours vs connected components

- **Connected components** label each pixel with its region id — a labelled image the size of the input.
- **Contours** give the ordered *boundary* of each region — a compact list of points.
- Use components to count/label fast; use contours to measure shape and crop. They pair naturally.

	Connected components	**Contours**
Output	labelled image (same size)	list of point sequences
Best for	counting, per-pixel stats	shape, boxes, cropping
OpenCV	`connectedComponents`	`findContours`

Key idea

Both turn a binary mask into a list of objects — the bridge from pixels to measurable things.

Basic measurements

The core measurements

Measurement	OpenCV
Area	`cv2.contourArea(c)`
Perimeter	`cv2.arcLength(c, True)`
Bounding box (axis-aligned)	`cv2.boundingRect(c)`
Rotated box	`cv2.minAreaRect(c)`
Enclosing circle	`cv2.minEnclosingCircle(c)`
Centroid	from `cv2.moments(c)`

These turn a shape into numbers you can *filter*, *sort*, *and compare* — the whole game this week.

Area — and the shoelace formula

Polygon area

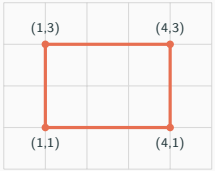
For a closed polygon with vertices (x_i, y_i):

$$A = \frac{1}{2}\left|\sum_{i=1}^{n}\left(x_i y_{i+1} - x_{i+1} y_i\right)\right|$$

(indices wrap: $x_{n+1} = x_1$). This is what `contourArea` computes from the boundary points.

- No need to count interior pixels — the boundary alone fixes the area.
- Green's theorem in disguise: a line integral around the loop gives the enclosed area.
- `contourArea` returns signed-then-absolute area; watch out for *very* thin contours ($A \approx 0$).

Worked example: area by shoelace



- Vertices in order: $(1,1), (4,1), (4,3), (1,3)$.
- $\sum x_i y_{i+1} = 1{\cdot}1 + 4{\cdot}3 + 4{\cdot}3 + 1{\cdot}1 = 26$.
- $\sum x_{i+1} y_i = 4{\cdot}1 + 4{\cdot}1 + 1{\cdot}3 + 1{\cdot}3 = 14$.
- $A = \frac{1}{2}|26 - 14| = \mathbf{6}$ — matches 3×2. ✓

Sanity check

Always cross-check a formula on a shape whose answer you already know.

Perimeter (arc length)

- `cv2.arcLength(c, closed=True)` sums the distances between consecutive contour points.
- For the box above: $3 + 2 + 3 + 2 = \mathbf{10}$ — the loop closes back to the start.
- **CHAIN_APPROX_SIMPLE** keeps perimeter accurate for straight edges; curves need more points.

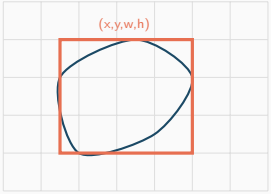
Perimeter

$P = \sum_{i=1}^{n} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}$

✗ Pitfall

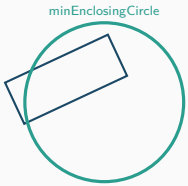
Pass `closed=False` by mistake and you drop the last edge — perimeter comes out too small.

Bounding box (axis-aligned)



- `x,y,w,h = cv2.boundingRect(c)` — smallest upright rectangle containing the contour.
- x, y = top-left corner; w, h = width, height. **Area** of box $= w \cdot h$.
- The crop `img[y:y+h, x:x+w]` is exactly one object, ready to hand off.
- Cheap, robust, and the workhorse of digit cropping.

Rotated box & enclosing circle



- `cv2.minAreaRect(c)` → (center, (w,h), angle): the smallest *rotated* rectangle.
- Use it when objects are tilted (a slanted meter, a rotated document).
- `cv2.minEnclosingCircle(c)` → (center, radius): handy for round parts / coins.

✔ Good practice

Rotated box width/height are orientation-independent — better for tilted digits than the upright box.

Centroid from moments

Centroid

From the image moments $M_{pq} = \sum_{x,y} x^p y^q \, I(x,y)$:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \qquad \bar{y} = \frac{M_{01}}{M_{00}}$$

M_{00} is the area (mass), M_{10}, M_{01} the first moments.

```python
M = cv2.moments(c)
cx = M["m10"] / M["m00"]
cy = M["m01"] / M["m00"]
```

- The centroid is the shape's "balance point."
- Robust anchor for tracking, ordering, and alignment.

Worked example: measure one digit

0	0	0	0
0	1	1	0
0	1	1	0
0	1	1	0
0	0	0	0

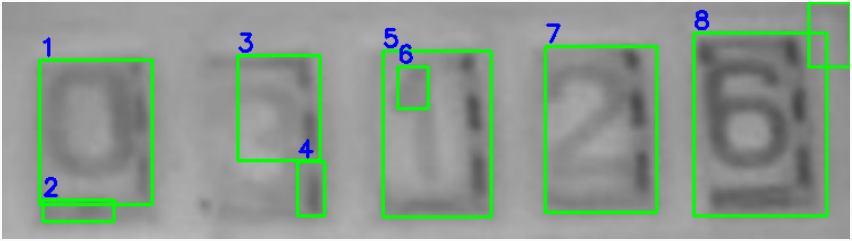
- White region: a 2×3 block of "1" pixels (columns 2–3, rows 2–4).
- **Area** ≈ 6 px (pixel count) — `contourArea` on the boundary gives $\approx$ the same.
- **Bounding box** $w = 2$, $h = 3 \Rightarrow$ box area $= 6$, **extent** $= 6/6 = 1.0$ (a solid rectangle).
- **Aspect ratio** $w/h = 2/3 \approx 0.67$ — taller than wide, digit-like.
- **Centroid** at the block's middle: $(\bar{x}, \bar{y}) = (2.5, 3.0)$.

Key idea

Six numbers now describe this blob — enough to decide "digit or noise?"

Boxes → crop, order, and filter

Bounding boxes → crop & order the digits



- Draw a box around each digit; **sort** the boxes by their x-coordinate (left to right).
- Crop each box → one clean image per digit, *in reading order*.
- This is exactly the DocScan hand-off to the recognizer (Week 13).

Cropping & ordering in code

```
boxes = [cv2.boundingRect(c) for c in contours]
boxes = sorted(boxes, key=lambda b: b[0])   # sort by x (left to right)
digits = [gray[y:y+h, x:x+w] for (x, y, w, h) in boxes]
```

- Sorting by x gives the number in reading order — no OCR needed yet.
- For multi-row layouts, sort by (row band, x): group by y first, then x within a row.

Tie-break

If two boxes share an x, add y as a secondary key: `key=lambda b:(b[0],b[1])`.

Filtering by shape

- Real images give junk contours — noise specks, partial marks, a long scratch.
- Filter using the numbers we just computed: **area** (too small = noise), **aspect ratio** (digits are tallish), **extent** (fill fraction of the box).
- A handful of thresholds removes most false positives before any recognition.

✔ Good practice

"Keep contours with area $>$ T and $0.2 < w/h < 1.0$" separates digits from noise robustly.

Worked example: which blobs survive the filter?

Blob	area	w	h	w/h	keep? (area>200, 0.2 <w/h< 1.2)
digit “7”	640	22	40	0.55	keep
noise speck	18	5	4	1.25	drop (area)
scratch	300	90	4	22.5	drop (w/h)
digit “1”	210	8	38	0.21	keep

- The speck fails on **area**; the scratch fails on **aspect ratio** — two rules, most junk gone.
- Thin “1” is a near-miss on w/h — this is why thresholds need a little slack.

Pitfalls when measuring

✕ Pitfall

A merged pair of digits looks like *one* big contour; a broken digit looks like *two*. Fix the mask upstream (Weeks 7–8) before trusting counts.

- `contourArea` of a self-intersecting or open contour is meaningless — keep contours closed.
- Bounding box in `(x,y,w,h)`; NumPy slicing is `[y:y+h, x:x+w]` — row first. Easy to swap.
- Tiny contours can have zero area $\Rightarrow$ division by $M_{00} = 0$ in the centroid. Filter first.

In-class exercise

In-class exercise (12 min)

Design the filter

You have contours from a meter photo: 9 digits plus some noise specks and one long scratch.

1. Which measurements distinguish a digit from a speck? From a scratch?
2. Write a filter rule (area + aspect ratio) that keeps digits and drops the rest.
3. After filtering, how do you output the digits in the correct left-to-right order?
4. What single upstream problem would break your count no matter how good the filter is?

Debrief

- Speck: tiny **area** → drop by `area > T`. Scratch: extreme **aspect ratio** (very wide/thin) → drop.
- Rule: keep if `area>200 and 0.2<w/h<1.2` — digits are compact and tallish.
- Order: sort the kept bounding boxes by x; read left to right.
- Upstream killer: merged/broken digits from a bad mask — no filter recovers a missing split.

Key idea

Shape measurements turn "which blobs are digits?" into a few numeric thresholds.

Wrap-up

What you can do after Session 1

- Find contours; pick a retrieval mode and an approximation method.
- Explain the difference between contours and connected components.
- Measure area (shoelace), perimeter, bounding box, rotated box, and centroid.
- Filter contours by area / aspect ratio / extent, then crop and order digits.

✔ Good practice

Every measurement here is one line of OpenCV — but knowing *which* number to threshold is the skill.

Next session

- We can measure a shape — but our numbers change if the object rotates or scales.
- Session 2: **shape descriptors** that are (nearly) invariant — circularity, solidity, moments, and **Hu moments** — plus convex hulls, polygon approximation, and contour hierarchy (holes).
- That's what lets DocScan find a page's 4 corners and match shapes without training.

Bring

Your filtered digit crops from today — Session 2 turns them into descriptors.

Contours turn regions into a list of objects.

Find, then measure: area, perimeter, box, centroid — the numbers everything else builds on.